

Architecture Document

Local-First Financial Analysis Application — V1

Status: Draft v3 (second-pass review revision) **Companion to:** docs/prd.md **Audience:** Engineers, technical reviewers, future contributors

Changes since v2: Resolves all findings from the second architecture review (live-verification pass). Notable changes:

- **Numeric storage** switched from SQLite `REAL` to `INTEGER` micro-units (×1,000,000) for all currency and per-share values, eliminating IEEE-754 drift in derived-metric subtraction (M9).
 - **Caching strategy** rewritten — SEC sends `Cache-Control: no-store` with no `ETag` / `Last-Modified`, so conditional-GET revalidation is dropped in favor of a local-TTL model (M12).
 - **superseded_by cycle protection** extended to the INSERT path (C1).
 - **Amendment-removes-a-concept** policy specified: original stays authoritative, partial-amendment caveat surfaced per period (C2).
 - **Item 4.02 resolution** moved from a fragile date-arithmetic predicate to an explicit `restatement_resolved_by` join table populated at amendment ingestion (M7).
 - **FX-rate source** switched to a bundled offline ECB historical-rate dataset, with refresh through a single allowlisted host — consistent with the local-first / minimize-outbound posture (M1, M5).
 - **historical_price** gains a `ticker` column to handle ticker changes (M6).
 - **Tauri 2 frontend protection** clarified — capabilities gate JS→Rust commands; WebView CSP `connect-src` gates frontend `fetch` /XHR; Rust-side `request` host allowlist gates Rust outbound. All three layers required (M3).
 - Plus assorted majors (M2 missing index added, M4 Yahoo language softened, M8 "latest shares" specified, M10 refinery + `foreign_keys` documented, M11 `synchronous=NORMAL` trade-off documented) and minors.
-

1. Introduction

1.1 Purpose

This document defines the technical architecture for V1 of a local-first macOS financial analysis application. It translates the product requirements in `docs/prd.md` into concrete technology choices, module boundaries, data structures, and runtime behavior. It is the source of truth for "how" the system is built; the PRD is the source of truth for "what" it must do.

1.2 Scope

V1 only. The architecture must accommodate the V2/V3 directions called out in the PRD (multi-company comparison, formula engine, export, plugins) without requiring a rewrite, but no V2+ feature is implemented in V1.

1.3 Key Architectural Drivers

Drawn directly from the PRD and ranked:

1. **Correctness** of financial data — the user must be able to trust every number.
2. **Traceability** — every displayed value must trace back to a specific SEC filing concept.
3. **Local-first** — no cloud backend, full offline operation after ingestion.

4. **Durability** — application crashes must not destroy ingested data.
5. **Modularity** — clean seams for future expansion (peer comparison, formula engine, export).
6. **Maintainability** — a small team / single developer should be able to evolve the codebase.
7. **Performance** — ingestion may be slow; navigation of ingested data must feel instant.

Speed of ingestion is explicitly the *lowest* priority among these.

Data acquisition note. V1's primary structured-data source is the SEC's pre-extracted XBRL facts JSON API at `data.sec.gov/api/xbrl/companyfacts`. *General* HTML / iXBRL parsing of 10-K and 10-Q narrative content is out of V1 scope. *Narrow*, accuracy-driven HTML parsing is in scope where it is the only path to a precise result — specifically, parsing 8-K Item 4.02 disclosures to identify the fiscal periods flagged as unreliable, and fetching historical EOD prices for market-cap-at-filing-date computation. See §3.5.

2. Architectural Style

The system is a **local-first, modular monolith** packaged as a single desktop application. There is no client/server split, no microservices, and no remote infrastructure.

Internally the codebase is organized as a layered architecture with a separate **pipeline architecture** for the ingestion subsystem:

- **Layered (UI → IPC → Domain Services → Persistence):** for read-side workflows, where the dashboard queries normalized data through repository interfaces.
- **Pipeline (Discover → Download → Parse → Normalize → Persist):** for the ingestion subsystem, where each stage is an isolated, testable unit with explicit inputs and outputs.

This split is deliberate: the read path optimizes for low-latency UI rendering, while the write path optimizes for correctness, observability, and resumability.

3. Technology Stack

3.1 Decision Summary

Concern	Choice	Alternative considered
Desktop shell	Tauri 2.x	Electron, Swift/AppKit
Backend language	Rust (in Tauri's <code>src-tauri/</code>)	Go, TypeScript (Node)
Frontend	React 18 + TypeScript + Vite	SwiftUI, Svelte
Styling	Tailwind CSS	CSS Modules, Stitches
Charting	Apache ECharts (via <code>echarts-for-react</code>)	Recharts, Chart.js
State / data fetching	TanStack Query over Tauri IPC	Redux, Zustand
Local database	SQLite via <code>rusqlite</code> (bundled)	DuckDB, Postgres
HTTP client	<code>request</code> with <code>rustls</code>	<code>ureq</code> , <code>hyper</code>

Concern	Choice	Alternative considered
Async runtime	<code>tokio</code>	<code>async-std</code>
Logging / tracing	<code>tracing</code> + <code>tracing-subscriber</code>	<code>log</code> + <code>env_logger</code>
Error handling	<code>thiserror</code> (lib) + <code>anyhow</code> (app boundary)	manual enums
Migrations	<code>refinery</code>	hand-rolled, <code>sqlx-migrate</code>
Build / package	Tauri bundler → <code>.dmg</code>	<code>cargo-bundle</code>

3.2 Rationale: Tauri over Electron

- **Bundle size & memory:** Tauri applications are typically ~3–10 MB and use the system WebView (WKWebView on macOS), versus Electron's ~80–150 MB Chromium runtime. For a single-user analytical app, the lighter footprint matters.
- **Security:** Tauri's IPC surface is opt-in per command, with an explicit allowlist. The PRD's "minimize third-party dependencies" and "avoid unnecessary outbound requests" principles map naturally onto Tauri's permission model.
- **Rust backend:** JSON parsing of SEC's `companyfacts` payload, taxonomy normalization, XBRL XML fallback parsing, narrow HTML parsing of 8-K Item 4.02 disclosures, market-data API clients, and SQLite work all benefit from Rust's correctness guarantees, error-handling discipline, and zero-cost abstractions. The normalization subsystem in particular is the kind of code where Rust's type system pays for itself.
- **Native macOS look-and-feel:** WKWebView integrates with macOS conventions (text rendering, scrolling, accessibility) more naturally than bundled Chromium.

3.3 Why not native Swift/AppKit?

- Significantly larger surface area for a single-developer effort.
- The financial-domain code (XBRL parsing, normalization) is not language-specific — keeping it in a portable Rust core preserves the option to ship Windows/Linux builds later without rewriting.
- The PRD explicitly lists Tauri as an acceptable approach.

3.4 Rationale: SQLite over DuckDB

DuckDB is more attractive for ad-hoc analytical queries, but for V1:

- The dataset is small (one company × ~80 quarters × ~200 facts ≈ 16 K rows; even 50 companies stays well under 1 M rows).
- SQLite has more mature tooling, ACID guarantees the PRD requires for durability, and a more stable on-disk format for long-term local storage.
- Tauri + `rusqlite` has a well-trodden integration path.
- DuckDB can be added later as a read-only analytical accelerator without changing the canonical store.

3.5 Rationale: how V1 acquires financial data from SEC EDGAR

This is the most important data-flow decision in V1. It is worth being precise about what is and is not on the wire.

10-K and 10-Q documents themselves are not in JSON. A filing is submitted to EDGAR as a bundle of HTML / inline XBRL (iXBRL) / XBRL XML and exhibits. There is no public API endpoint that returns "the 10-K, as JSON." Anyone parsing a 10-K is parsing HTML or XML.

What V1 actually consumes is a different artifact. Since the SEC XBRL mandate (large filers ~2009, all U.S.-listed filers ~2011), every numeric line item on the financial statements in a 10-K, 10-Q, or financial-statement 8-K must be tagged with a structured XBRL concept (e.g., `us-gaap:Revenues` , `us-gaap:Assets` , `us-gaap:NetIncomeLoss`). The SEC ingests those tagged values into a structured database and exposes a public read-side at:

```
https://data.sec.gov/api/xbrl/companyfacts/CIK{cik10}.json
```

This JSON is **not the 10-K**. It is the SEC's own pre-extracted view of every XBRL fact the company has ever tagged in a 10-K / 10-Q / 8-K, aggregated into a single JSON document keyed by taxonomy and concept. Each fact in the response carries:

Field	Meaning
<code>val</code>	Numeric value in the declared unit (USD facts are integer-valued at the source)
<code>unit</code> (key under <code>units</code>)	<code>USD</code> , <code>shares</code> , <code>USD/shares</code> , etc.
<code>start</code> , <code>end</code>	Period covered (<code>end</code> only for instant facts)
<code>fy</code>	Fiscal year
<code>fp</code>	Fiscal period (<code>FY</code> , <code>Q1</code> , <code>Q2</code> , <code>Q3</code> , <code>Q4</code>)
<code>form</code>	Source form type (<code>10-K</code> , <code>10-Q</code> , <code>10-K/A</code> , <code>8-K</code> , ...)
<code>accn</code>	Accession number of the source filing
<code>filed</code>	ISO date the filing was submitted
<code>frame</code>	Calendar-period identifier (e.g., <code>CY2016</code>) when applicable

The response is keyed by taxonomy (`us-gaap` , `dei` , and occasionally industry-specific or filer-custom taxonomies). The `dei` taxonomy carries entity-level facts such as `EntityReportingCurrencyISOCode` (used by §8.3 for non-USD filers) and `DocumentPeriodEndDate` .

The `accn` is what gives V1 its source-filing traceability for free (FR-060): every normalized value can be linked back to the exact filing it came from without us touching the filing itself.

What V1 gets from `companyfacts` : every tagged numeric line item on the income statement, balance sheet, and cash-flow statement, across every reporting period the company has filed, for any U.S.-listed company that complies with the XBRL mandate. This is exactly the data the V1 PRD requires (FR-020 / FR-021 / FR-022).

What V1 does *not* get from `companyfacts` :

- MD&A narrative
- Footnote prose
- Risk factors and other Reg S-K narrative sections
- Exhibits in their entirety
- Anything inside the 10-K that is not XBRL-tagged

General HTML parsing of 10-K narrative content — to support summarization, sentiment analysis, "what did management say" — is not in V1 scope. The PRD defers that class of capability to V3.

Narrow, accuracy-driven HTML parsing IS in V1 scope. Per the PRD's correctness driver and FR-011 (which explicitly lists "HTML/text parsing fallback" as a permitted source), V1 includes two targeted HTML/text parsing paths. Both are

narrow, well-bounded, and motivated by data-integrity rather than feature breadth:

- 1. **8-K Item 4.02 disclosures** — parsed to identify the specific fiscal periods the company is flagging as unreliable. The system must reliably extract those period identifiers regardless of phrasing or whether the filing carries structured tags. Result is stored in a dedicated `restatement_announcement` table (see §6.3) and used to surface a per-period warning in the dashboard, which clears once amendments covering the named periods are filed. **No financial values are extracted from 8-Ks.** See §6.3, §8.7, §9.1.
- 2. **Historical EOD prices** for market-cap-at-filing-date computation. Per FR-050, V1 must show historical market cap at each filing date offline; this is computed at ingestion time as $\text{price} \times \text{shares-outstanding-from-companyfacts}$ and persisted locally. Price data comes from a separate market-data adapter (see §7.5), not from EDGAR.

Summary of V1 data sources:

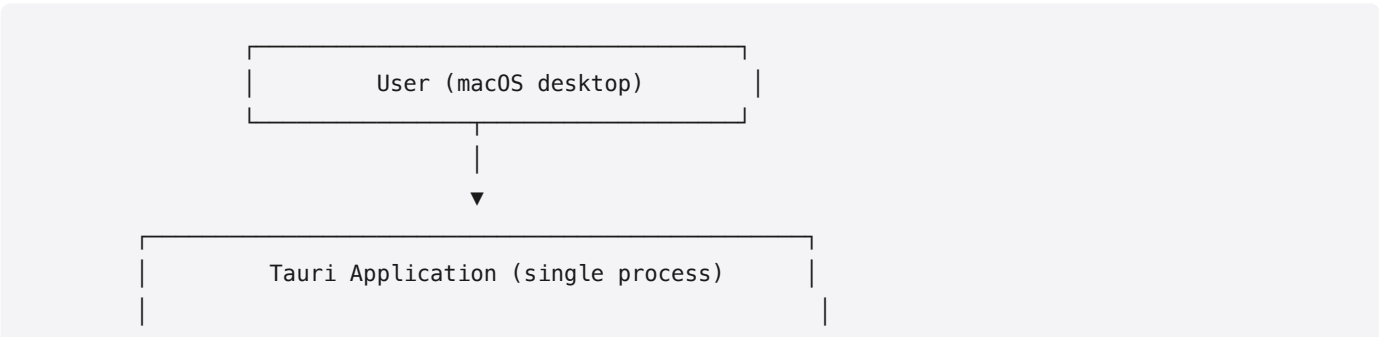
Source	Format	Used for	Path
<code>data.sec.gov/api/xbrl/companyfacts/CIK*.json</code>	JSON	Structured financial facts (primary)	Normal
<code>data.sec.gov/submissions/CIK*.json</code>	JSON	Filing index, including 8-K item lists	Normal
Raw XBRL instance document	XML	Targeted re-fetch of values missing from <code>companyfacts</code>	Fallback
8-K primary document (Item 4.02 only)	HTML / iXBRL	Identifying restated periods	Normal (when an Item 4.02 8-K is in the submissions index)
Market-data adapter	JSON (provider-specific)	Historical EOD prices for market cap	Normal

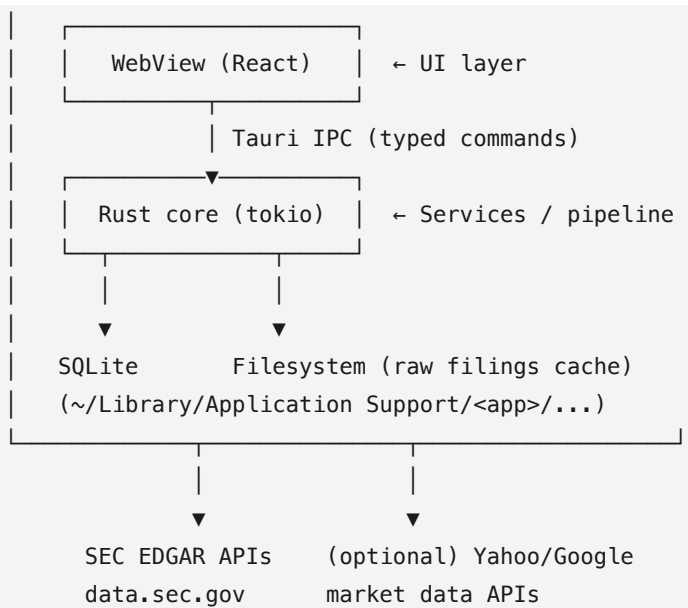
What is still excluded from V1: parsing 10-K / 10-Q HTML narrative, parsing any 8-K item other than Item 4.02, downloading filing exhibits, and any text-extraction work whose output is not a structured fact backed by full traceability.

The fallback is implemented behind the same `FactSource` trait as the JSON path, so it can be enabled per-concept-per-filing without disturbing the rest of the pipeline.

4. High-Level Architecture

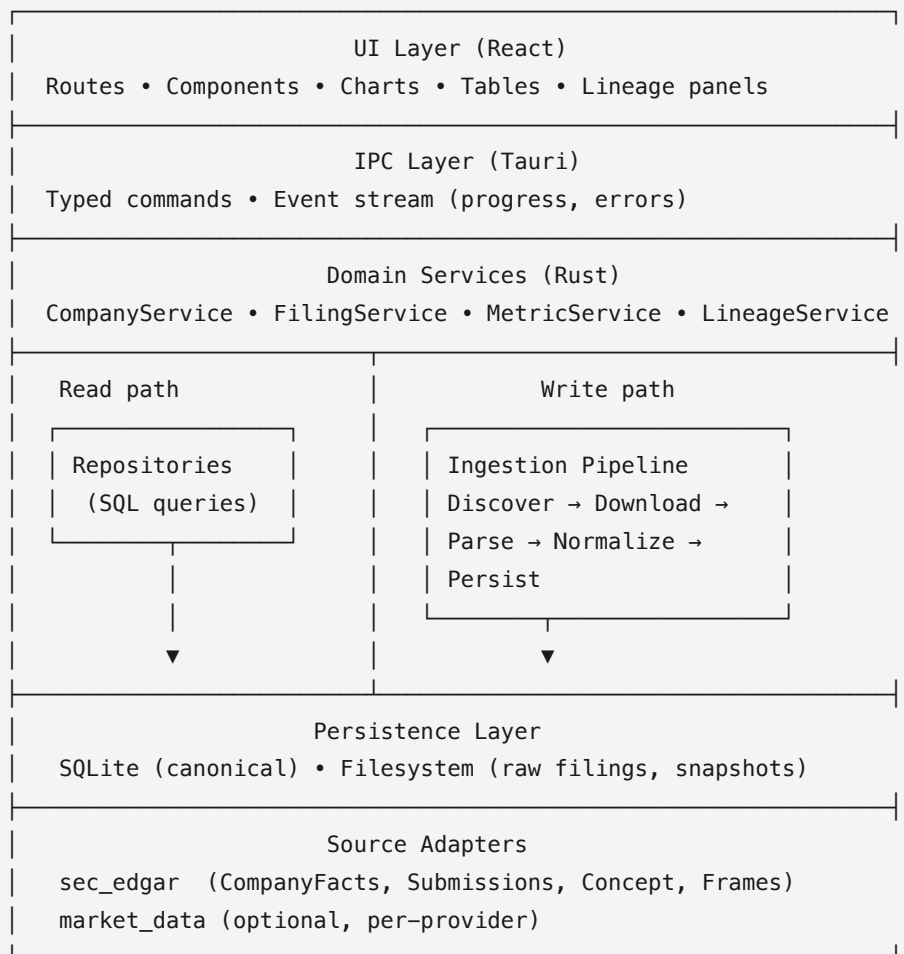
4.1 System Context





There is exactly one outbound dependency that is required for ingestion: the SEC EDGAR APIs. Market-data APIs are optional and the application degrades gracefully without them.

4.2 Layer Diagram



The strict downward dependency rule applies: each layer depends only on layers below it. The UI never reaches into the database directly; the ingestion pipeline never calls into the UI; source adapters never know about SQL.

5. Module Structure

5.1 Repository layout

```
econ_project/
├─ docs/                # PRD, architecture, future ADRs
├─ src/                 # React app
│  ├─ api/              #   Typed Tauri IPC client
│  ├─ features/
│  │  ├─ home/          #   Saved companies, add ticker
│  │  ├─ company/       #   Dashboard, charts, tables
│  │  ├─ lineage/       #   Drill-down to filing source
│  │  └─ ingestion/     #   Progress UI
│  ├─ components/      #   Generic UI primitives
│  ├─ charts/          #   ECharts wrappers
│  ├─ state/           #   TanStack Query hooks
│  └─ styles/
├─ src-tauri/           # Rust core
│  ├─ src/
│  │  ├─ main.rs
│  │  ├─ ipc/           #   #[tauri::command] entry points
│  │  ├─ domain/       #   Service layer (use-cases)
│  │  ├─ ingestion/    #   Pipeline stages
│  │  │  ├─ discover.rs
│  │  │  ├─ download.rs
│  │  │  ├─ parse.rs
│  │  │  ├─ normalize.rs
│  │  │  └─ persist.rs
│  │  ├─ normalization/ #   Concept maps, unit/period rules
│  │  │  ├─ concept_map.rs
│  │  │  ├─ periods.rs
│  │  │  ├─ units.rs
│  │  │  └─ signs.rs
│  │  ├─ sources/
│  │  │  ├─ sec_edgar/  #   companyfacts/submissions JSON, XBRL XML fallback,
│  │  │  │              #   Item 4.02 8-K HTML parser
│  │  │  └─ market_data/ #   MarketDataAdapter trait + Yahoo Finance default impl
│  │  ├─ persistence/
│  │  │  ├─ db.rs      #   Connection pool, migrations
│  │  │  ├─ repositories/ #   One module per aggregate
│  │  │  └─ schema/    #   .sql migrations
│  │  ├─ derived/      #   Derived-metric formulas
│  │  ├─ lineage/      #   Source-tracking helpers
│  │  └─ errors.rs
```

```

| | └─ telemetry.rs          # tracing setup (local logs only)
| └─ Cargo.toml
└─ README.md

```

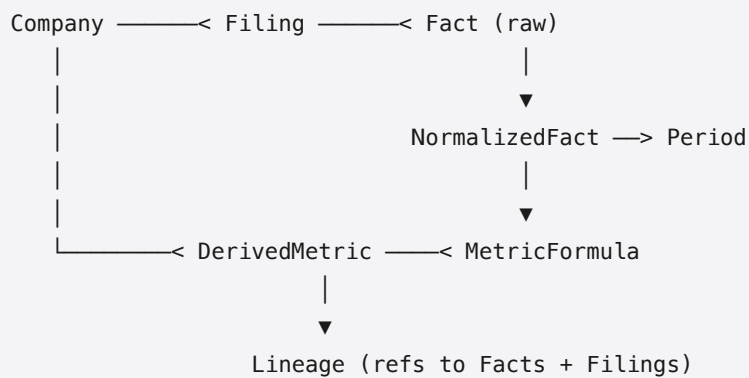
5.2 Crate / module boundaries

The Rust code is one binary crate but organized so that each top-level module under `src-tauri/src/` could be extracted into its own crate later without churn. In particular:

- `normalization/` depends on no I/O, only on domain types — it is pure logic and should be heavily unit-tested.
- `sources/sec_edgar/` depends on `request` and the raw EDGAR JSON shape, but exposes only typed `RawFact` / `RawFiling` records to the rest of the system.
- `persistence/repositories/` is the only module allowed to write SQL.

6. Data Architecture

6.1 Conceptual model



6.2 Canonical metric model

The product's correctness hinges on a stable internal vocabulary. V1 ships a hard-coded **canonical metric catalog** — a small enum of well-defined financial concepts with documented semantics:

Canonical metric	Statement	Aggregation	Origin	Sign convention
revenue	Income	Period flow	XBRL	Positive
cost_of_revenue	Income	Period flow	XBRL	Positive
gross_profit	Income	Period flow	XBRL or derived (revenue – cost_of_revenue)	Positive
operating_income	Income	Period flow	XBRL	Signed
net_income	Income	Period flow	XBRL	Signed
eps_basic	Income	Period flow	XBRL	Signed

Canonical metric	Statement	Aggregation	Origin	Sign convention
eps_diluted	Income	Period flow	XBRL	Signed
shares_outstanding_basic	Income	Period instant	XBRL	Positive
shares_outstanding_diluted	Income	Period instant	XBRL	Positive
cash_and_equivalents	Balance	Instant	XBRL	Positive
long_term_debt	Balance	Instant	XBRL	Positive
current_debt	Balance	Instant	XBRL	Positive
total_debt	Balance	Instant	Derived (long_term_debt + current_debt)	Positive
total_assets	Balance	Instant	XBRL	Positive
total_liabilities	Balance	Instant	XBRL	Positive
total_equity	Balance	Instant	XBRL	Signed
cash_from_operations	Cash flow	Period flow	XBRL	Signed
capital_expenditures	Cash flow	Period flow	XBRL or derived (Δ PP&E_net + depreciation_and_amortization)	Stored positive (sign-normalized)
depreciation_amortization	Cash flow	Period flow	XBRL	Positive
free_cash_flow	Cash flow	Period flow	Derived (net_income + depreciation_amortization - capital_expenditures)	Signed
property_plant_and_equipment_net	Balance	Instant	XBRL	Positive
operating_margin	Income (ratio)	Period flow	Derived (operating_income ÷ revenue)	Signed ratio (x1e6)
net_interest_income , noninterest_income , interest_income_operating , interest_expense	Income	Period flow	XBRL (bank inputs)	Signed
historical_market_cap	Market	Instant (at filing date)	Derived (historical_price_at_filed_at × shares_outstanding_basic)	Positive
current_market_cap	Market	Live	Derived (live_price × latest_shares_outstanding_basic)	Positive

Each entry records its statement, whether it is a flow (period) or stock (instant), how it originates, and the canonical sign convention used at storage time. UI display rules can then invert signs consistently for presentation (e.g., showing CapEx as a negative cash outflow on a cash-flow waterfall).

total_debt definition. XBRL has no single canonical "total debt" concept. V1 defines `total_debt = long_term_debt + current_debt`, where the inputs map to a primary-then-fallback chain of XBRL concepts:

- `long_term_debt` ← `us-gaap:LongTermDebt`, falling back to `us-gaap:LongTermDebtNoncurrent` when the primary is absent.
- `current_debt` ← `us-gaap:DebtCurrent`, falling back to `us-gaap:LongTermDebtCurrent`, then `us-gaap:ShortTermBorrowings`.

`total_debt` is **derived at read time** rather than persisted: a stale `current_debt` that has since been superseded would otherwise leave the persisted sum out of sync with its inputs. The IPC read path joins the most recent primary `long_term_debt` and `current_debt` per period and sums them on the fly. The formula and the resolved input concepts are surfaced in the lineage panel for transparency (FR-031). The same read-time strategy applies to `gross_profit`, `capital_expenditures` (when derived from the PP&E roll-forward), `free_cash_flow`, and `operating_margin`, described in §10.

free_cash_flow and operating_margin. Both are multi-input derivations computed at read time for the same supersession-safety reason as `total_debt`. `free_cash_flow = net_income + depreciation_amortization - capital_expenditures` (capital expenditures is stored positive, hence subtracted); all three inputs are required for a period, and capital expenditures uses the full read-time series so the PP&E-roll-forward fallback flows through. `operating_margin = operating_income ÷ revenue` is a dimensionless ratio stored $\times 1e6$ (per the numeric-storage convention below); revenue uses the read-time series (with the bank-revenue fallback), the margin is omitted when revenue is non-positive, and an operating loss yields a valid negative margin. The pure formulas live in the `derived` module (heavily unit-tested in isolation) and the per-period series assembly in `derived::series`, which is parameterized over the repository traits so the IPC handlers and the production-mode integration tests share one code path.

Bank revenue. Bank-holding filers do not generally tag `us-gaap:Revenues`. Architecture §8.1 documents the resolution chain (NetInterestIncome+NoninterestIncome, then InterestIncomeOperating-InterestExpense+NoninterestIncome). The bank-revenue derivation runs once at ingest time per period that has no direct `Revenue` value, and is persisted to `derived_metric` with `formula_id = "bank_revenue_v1"`. Read-time `Revenue` queries union the direct values with the bank-revenue derivations so the dashboard widget never falls back to an empty series for a financial company.

Market-cap metrics.

- `historical_market_cap` is computed once per filing at ingestion time and persisted. Its inputs are the historical EOD price on the filing's `filed_at` date (from a market-data adapter — see §7.5) and `shares_outstanding_basic` from the same filing. It is offline-available because it is fully persisted.
- `current_market_cap` is computed on demand from a live price source. When the live source is unavailable (offline or the source is down), the dashboard widget renders an explicit "current market cap unavailable" state and the historical series remains visible.

No TTM in V1. An earlier draft proposed a TTM aggregation. TTM is not a PRD requirement, and the PRD's annual/quarterly chart toggle (FR-051) already addresses the long-horizon analytical need. V1 defers TTM; the dashboard summary widgets show the latest reported annual value with a sparkline of recent quarters instead of a synthetic TTM.

Numeric storage convention. All currency and per-share values are stored as `INTEGER` **micro-units** — the value in the base currency, multiplied by 1,000,000, stored as a 64-bit integer. SEC's `companyfacts` returns USD facts as integer-valued JSON numbers (e.g., `"val": 215639000000`); storing them as IEEE-754 doubles would silently introduce rounding in derived-metric subtractions like `gross_profit = revenue - cost_of_revenue`. Micro-units give us exact arithmetic for all values that fit in `i64` ($\pm 9.2 \times 10^{18}$ micro-units = $\pm 9.2 \times 10^{12}$ base units = ± 9.2 trillion dollars), which is well above any plausible value. Per-share metrics (EPS, ratios) use the same convention: \$1.234567 EPS is `1234567`. Conversion to display units (dollars, share counts) happens in the UI layer.

Share counts are also stored as `INTEGER`, but as absolute integer share counts (no scaling) since shares are inherently integer.

The catalog is a Rust enum so missing values are caught at compile time. New metrics are added by extending the enum, the concept map, and (where applicable) a derived formula.

6.3 SQLite schema (V1)

```
-- Companies
CREATE TABLE company (
  cik            TEXT PRIMARY KEY,          -- 10-digit zero-padded
  ticker         TEXT NOT NULL,
  name           TEXT NOT NULL,
  exchange       TEXT,
  sic            TEXT,
  fiscal_year_end TEXT,                    -- MMDD
  added_at       TEXT NOT NULL,           -- ISO8601
  last_refreshed TEXT
);
CREATE UNIQUE INDEX idx_company_ticker ON company(ticker);

-- Filings (one row per accession number per company)
CREATE TABLE filing (
  accession_no    TEXT PRIMARY KEY,        -- e.g. "0000320193-24-000123"
  cik            TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
  form_type       TEXT NOT NULL,           -- 10-K, 10-Q, 10-K/A, 10-Q/A, 8-K, ...
  filed_at       TEXT NOT NULL,           -- ISO date
  period_of_report TEXT,                  -- ISO date
  is_amendment    INTEGER NOT NULL DEFAULT 0,
  amends         TEXT,                    -- accession_no this amends
  item_4_02_8k   INTEGER NOT NULL DEFAULT 0, -- 1 iff form_type='8-K' AND items contains '4.02'
  source_url      TEXT,
  raw_path        TEXT                    -- local file path if cached
);
CREATE INDEX idx_filing_cik_filed ON filing(cik, filed_at DESC);

-- (Note: the `filing` CREATE above includes `item_4_02_8k INTEGER NOT NULL DEFAULT 0`,
-- set to 1 iff form_type='8-K' AND items contains '4.02'. Shown inline in the migration.)

-- Periods (canonicalized fiscal periods).
-- fiscal_quarter uses 0 for annual rows (NOT NULL) so the UNIQUE
-- constraint actually constrains annual periods. kind is derived
-- from fiscal_quarter via CHECK and stored for query convenience.
CREATE TABLE period (
  id              INTEGER PRIMARY KEY,
  cik            TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
  fiscal_year     INTEGER NOT NULL,
  fiscal_quarter  INTEGER NOT NULL,       -- 0 = annual, 1..4 = quarterly
  fiscal_year_end TEXT NOT NULL,         -- MMDD; lets us detect FYE changes
```

```

start_date      TEXT NOT NULL,          -- ISO date
end_date        TEXT NOT NULL,          -- ISO date
kind            TEXT NOT NULL,          -- 'annual' | 'quarterly'
is_53_week      INTEGER NOT NULL DEFAULT 0, -- 1 for 53-week fiscal years (retailers)
CHECK (fiscal_quarter BETWEEN 0 AND 4),
CHECK ((fiscal_quarter = 0 AND kind = 'annual') OR
      (fiscal_quarter BETWEEN 1 AND 4 AND kind = 'quarterly')),
UNIQUE (cik, fiscal_year, fiscal_quarter)
);
CREATE INDEX idx_period_cik_year ON period(cik, fiscal_year);

-- Raw facts (1:1 with what we got from EDGAR / XBRL).
-- Stored as INTEGER scaled by the $6.2 micro-unit convention:
--   USD          → ×1,000,000 (e.g., $215.639B = 215_639_000_000_000)
--   shares       → ×1          (absolute integer share count)
--   USD/shares   → ×1,000,000 (e.g., $1.234567 = 1_234_567)
--   pure        → ×1,000,000 (decimal ratios)
-- The companyfacts JSON path always receives absolute values from SEC;
-- scaling happens at parse time before insert. The XBRL XML fallback
-- applies its own decimals/unitRef scaling at parse time.
CREATE TABLE raw_fact (
  id          INTEGER PRIMARY KEY,
  cik         TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
  accession_no TEXT NOT NULL REFERENCES filing(accession_no) ON DELETE RESTRICT,
  taxonomy    TEXT NOT NULL,          -- 'us-gaap', 'dei', 'ifrs-full', ...
  concept     TEXT NOT NULL,          -- e.g. 'Revenues'
  unit        TEXT NOT NULL,          -- 'USD', 'shares', 'USD/shares', ...
  value_numeric INTEGER NOT NULL,      -- scaled per $6.2 storage convention
  period_start TEXT,                  -- NULL iff is_instant=1
  period_end  TEXT NOT NULL,          -- end date for instants
  is_instant  INTEGER NOT NULL,
  fy          INTEGER,
  fp          TEXT,                  -- 'FY','Q1','Q2','Q3','Q4'
  filed       TEXT,                  -- ISO date the source filing was submitted
  source_kind TEXT NOT NULL,          -- 'xbrl_api' | 'xbrl_xml'
  ingested_at TEXT NOT NULL,
  -- Natural-key UNIQUE so refresh re-ingestion is idempotent without
  -- creating duplicate raw_fact rows.
  UNIQUE (cik, accession_no, taxonomy, concept, unit, period_start, period_end, fp)
);
CREATE INDEX idx_raw_cik_concept ON raw_fact(cik, taxonomy, concept);
CREATE INDEX idx_raw_filing ON raw_fact(accession_no);

-- Canonical / normalized facts.
-- Multiple alternates may exist per (cik, metric, period_id); exactly
-- one carries is_primary=1. The dashboard reads only primary, current
-- (not superseded) rows. Currency / per-share values are stored as
-- INTEGER micro-units per $6.2; share counts as absolute integers.
-- For non-USD source facts, original_value/original_unit/fx_*

```

```

-- columns preserve the as-reported value alongside the USD-converted
-- value (NULL when the source unit is already USD).
CREATE TABLE normalized_fact (
  id            INTEGER PRIMARY KEY,
  cik           TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
  metric        TEXT NOT NULL,           -- canonical metric name
  period_id     INTEGER NOT NULL REFERENCES period(id) ON DELETE RESTRICT,
  value         INTEGER NOT NULL,        -- scaled per §6.2
  unit          TEXT NOT NULL,           -- 'USD', 'shares', 'USD/shares'
  source_fact_id INTEGER NOT NULL REFERENCES raw_fact(id) ON DELETE RESTRICT,
  source_kind    TEXT NOT NULL,          -- 'xbrl_api' | 'xbrl_xml'
  is_primary     INTEGER NOT NULL DEFAULT 1, -- 1 = canonical chosen value, 0 = alternate
  -- FX-conversion lineage (NULL when the source unit is already USD)
  original_value INTEGER,                -- as-reported value, scaled per §6.2
  original_unit   TEXT,                  -- e.g., 'EUR', 'JPY'
  fx_rate_micro   INTEGER,               -- conversion rate × 1,000,000 to USD
  fx_rate_source  TEXT,                  -- 'ECB-bundled' | 'ECB-online' | 'manual'
  fx_rate_date    TEXT,                  -- ISO date of the rate used
  -- superseded_by is a linked list: each prior value points at its
  -- IMMEDIATE successor (not a flat pointer to the latest). Walking
  -- the chain reconstructs the full restatement history. Cycles are
  -- prevented by the triggers declared below (both INSERT and UPDATE
  -- paths).
  superseded_by  INTEGER REFERENCES normalized_fact(id) ON DELETE RESTRICT,
  ingested_at    TEXT NOT NULL,
  UNIQUE (cik, metric, period_id, source_fact_id)
);
-- Exactly one primary, non-superseded row per (cik, metric, period_id):
CREATE UNIQUE INDEX idx_norm_primary_current
  ON normalized_fact (cik, metric, period_id)
  WHERE is_primary = 1 AND superseded_by IS NULL;
CREATE INDEX idx_norm_cik_metric_period ON normalized_fact(cik, metric, period_id);
-- Inverse-supersession walk for the lineage panel.
CREATE INDEX idx_norm_superseded_by
  ON normalized_fact(superseded_by)
  WHERE superseded_by IS NOT NULL;

-- Cycle protection on supersession chain – both UPDATE and INSERT paths.
-- The body is duplicated because SQLite does not support multi-event
-- triggers; both triggers walk the chain forward from NEW.superseded_by
-- and abort if NEW.id appears.
CREATE TRIGGER trg_norm_no_cycle_update
  BEFORE UPDATE OF superseded_by ON normalized_fact
  WHEN NEW.superseded_by IS NOT NULL AND EXISTS (
    WITH RECURSIVE chain(id) AS (
      SELECT NEW.superseded_by
      UNION ALL
      SELECT nf.superseded_by
      FROM normalized_fact nf JOIN chain ON nf.id = chain.id

```

```

        WHERE nf.superseded_by IS NOT NULL
    )
    SELECT 1 FROM chain WHERE id = NEW.id
)
BEGIN
    SELECT RAISE(ABORT, 'normalized_fact.superseded_by would create a cycle');
END;

CREATE TRIGGER trg_norm_no_cycle_insert
BEFORE INSERT ON normalized_fact
WHEN NEW.superseded_by IS NOT NULL AND EXISTS (
    WITH RECURSIVE chain(id) AS (
        SELECT NEW.superseded_by
        UNION ALL
        SELECT nf.superseded_by
        FROM normalized_fact nf JOIN chain ON nf.id = chain.id
        WHERE nf.superseded_by IS NOT NULL
    )
    SELECT 1 FROM chain WHERE id = NEW.id
)
BEGIN
    SELECT RAISE(ABORT, 'normalized_fact.superseded_by would create a cycle (insert)');
END;

-- 8-K Item 4.02 restatement announcements: which periods are flagged
-- unreliable by which 8-K filing. Resolved via restatement_resolved_by
-- below, populated at amendment-ingestion time (see §8.7).
CREATE TABLE restatement_announcement (
    id                INTEGER PRIMARY KEY,
    cik               TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
    accession_no      TEXT NOT NULL REFERENCES filing(accession_no) ON DELETE RESTRICT,
    affected_period_id INTEGER NOT NULL REFERENCES period(id) ON DELETE RESTRICT,
    filed_at          TEXT NOT NULL,
    ingested_at       TEXT NOT NULL,
    UNIQUE (accession_no, affected_period_id)
);
CREATE INDEX idx_restate_cik ON restatement_announcement(cik);

-- Resolution join: an Item 4.02 announcement is "resolved" for a given
-- period when an amendment covering that period has been ingested.
-- Populated by the persist worker when an amendment lands.
CREATE TABLE restatement_resolved_by (
    restatement_announcement_id INTEGER NOT NULL
        REFERENCES restatement_announcement(id) ON DELETE RESTRICT,
    resolving_accession_no TEXT NOT NULL
        REFERENCES filing(accession_no) ON DELETE RESTRICT,
    resolved_at           TEXT NOT NULL,
    PRIMARY KEY (restatement_announcement_id, resolving_accession_no)
);

```

```

-- Amendment coverage gaps: when an amendment is ingested but does NOT
-- re-tag a concept the original filing tagged for the same period, the
-- original normalized_fact row stays primary (per §8.5) and we record
-- a row here so the dashboard can surface a per-period caveat.
CREATE TABLE amendment_coverage_gap (
  id          INTEGER PRIMARY KEY,
  cik         TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
  amendment_accession_no TEXT NOT NULL
    REFERENCES filing(accession_no) ON DELETE RESTRICT,
  metric      TEXT NOT NULL,
  period_id   INTEGER NOT NULL REFERENCES period(id) ON DELETE RESTRICT,
  ingested_at TEXT NOT NULL,
  UNIQUE (amendment_accession_no, metric, period_id)
);
CREATE INDEX idx_amend_gap_cik_period ON amendment_coverage_gap(cik, period_id);

-- Historical EOD prices. Stored per (cik, date) but with the ticker
-- in effect at fetch time recorded so ticker changes (FB-META) don't
-- corrupt lineage. close_micro is USD × 1,000,000 (e.g., $190.45 = 190450000).
CREATE TABLE historical_price (
  cik         TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
  date        TEXT NOT NULL,           -- ISO date
  ticker      TEXT NOT NULL,           -- ticker in effect at fetch time
  close_micro INTEGER NOT NULL,         -- USD × 1,000,000
  source      TEXT NOT NULL,           -- adapter id
  ingested_at TEXT NOT NULL,
  PRIMARY KEY (cik, date)
);

-- Bundled offline FX-rate dataset (ECB historical reference rates).
-- Refreshed via one allowlisted host (see §7.5). rate_micro is the
-- conversion rate to USD × 1,000,000 (e.g., EUR-to-USD 1.0834 = 1083400).
CREATE TABLE fx_rate (
  currency    TEXT NOT NULL,           -- ISO 4217 code, e.g. 'EUR'
  date        TEXT NOT NULL,           -- ISO date
  rate_micro  INTEGER NOT NULL,         -- × 1,000,000 to USD
  source      TEXT NOT NULL,           -- 'ECB-bundled' | 'ECB-online'
  PRIMARY KEY (currency, date)
);

-- Derived metrics (computed from normalized facts and/or prices).
-- value is in INTEGER micro-units (per §6.2) when defined; NULL when
-- is_complete = 0.
CREATE TABLE derived_metric (
  id          INTEGER PRIMARY KEY,
  cik         TEXT NOT NULL REFERENCES company(cik) ON DELETE RESTRICT,
  formula_id  TEXT NOT NULL,           -- 'fcf_v1', 'total_debt_v1', 'historical_market_cap_v1'
  period_id   INTEGER NOT NULL REFERENCES period(id) ON DELETE RESTRICT,

```

```

value            INTEGER,
is_complete      INTEGER NOT NULL,          -- 0 if any input missing
computed_at      TEXT NOT NULL,
UNIQUE (cik, formula_id, period_id)
);

-- Ingestion diagnostics (observable to the user).
CREATE TABLE ingestion_event (
  id              INTEGER PRIMARY KEY,
  cik             TEXT REFERENCES company(cik) ON DELETE RESTRICT,
  accession_no    TEXT REFERENCES filing(accession_no) ON DELETE RESTRICT,
  stage          TEXT NOT NULL,             -- discover|download|parse|normalize|persist|item_4_02|
  level          TEXT NOT NULL,             -- info|warn|error
  user_visible    INTEGER NOT NULL DEFAULT 0, -- 1 if also surfaced on the dashboard, not only Diagno
  message        TEXT NOT NULL,
  detail_json     TEXT,
  occurred_at     TEXT NOT NULL
);
CREATE INDEX idx_ing_cik_time ON ingestion_event(cik, occurred_at DESC);

```

Notes:

- `raw_fact` and `normalized_fact` are kept as **separate tables** rather than overwriting raw data. This is what makes traceability cheap: a normalized row points at exactly one raw row, which points at exactly one filing.
- `superseded_by` is a **linked list**, not a flat pointer. When amendment A2 supersedes A1 (which already superseded the original O), the chain is `0 → A1 → A2`. The current value is whichever row has `superseded_by IS NULL`. The lineage panel walks the chain backward (via `idx_norm_superseded_by`) to show the full restatement history.
- Cycle protection on `superseded_by` is enforced by **two** SQLite triggers — `trg_norm_no_cycle_update` and `trg_norm_no_cycle_insert` — covering both the documented update path (§8.5) and any insert path that might set `superseded_by` directly (migrations, future fix scripts).
- **Amendment coverage gaps** are not silent. When an amendment omits a concept the original tagged, the original row stays primary and an `amendment_coverage_gap` row is inserted; the dashboard surfaces a per-period caveat for affected periods (§8.5, §11.2).
- All child tables have explicit `cik` FKs. The default `ON DELETE RESTRICT` policy enforces FR-004 (removal preserves cached data unless explicitly cleared) — actually deleting a company will fail until an explicit "uncache" operation drops dependent rows.
- Confidence scoring is deferred to V2. The V1 alternates model uses a binary `is_primary` flag instead; the lineage panel surfaces alternate facts on demand and Diagnostics tracks the resolution decision.
- All currency / per-share values use `INTEGER` micro-units (×1,000,000) per §6.2; share counts use absolute integers; FX rates use `INTEGER` micro-rate (×1,000,000). No `REAL` -typed financial values exist in the schema.
- The schema is migration-managed; **V1 uses additive migrations only** (no `DROP COLUMN` / table rebuilds). Refinery runs each migration in a transaction, and `PRAGMA foreign_keys` cannot be toggled mid-transaction (per SQLite docs); any future schema evolution that requires a rebuild will need to run outside refinery's transaction model. Documented here so future contributors don't trip on it.

6.4 Filesystem layout


```

~/Library/Application Support/com.<vendor>.econ-project/
├─ data.sqlite                # canonical DB
├─ data.sqlite-wal            # WAL (journal_mode=WAL)
├─ filings/
│   └─ <cik>/
│       ├── companyfacts.json # SEC's pre-extracted XBRL facts, one per company
│       ├── submissions.json  # SEC filing index, one per company
│       ├── prices.json       # cached historical EOD price series
│       └─ <accession_no>/
│           ├── primary_doc.xml # raw XBRL instance document (XBRL-XML fallback only)
│           ├── item_4_02.html  # 8-K Item 4.02 primary document (Item 4.02 8-Ks only)
│           └─ metadata.json
├─ reference/
│   └─ fx/
│       ├── ECB-historical-rates-bundled.csv # Shipped with the app
│       └─ ECB-updates-<date>.csv          # Incremental updates fetched on refresh
├─ logs/
│   └─ econ-project.<date>.log
└─ config.json                  # user prefs

```

The three top-level files under each `<cik>/` directory (`companyfacts.json` , `submissions.json` , `prices.json`) are the per-company artifacts V1 fetches in the normal path. The per- `<accession_no>/` subdirectory is created only for filings that require per-filing artifacts: an Item 4.02 8-K (HTML document needed for period extraction) or a `companyfacts` gap that triggered the XBRL-XML fallback. For most companies, that subdirectory is empty or absent.

Raw payloads are kept on disk (not in SQLite blobs) so they can be inspected, backed up, or re-processed without touching the database. The `raw_path` column in `filing` points to the relevant directory or file.

7. SEC Integration

7.1 Endpoints used

Purpose	Endpoint	Notes
Ticker → CIK lookup	<code>https://www.sec.gov/files/company_tickers.json</code>	Cached locally; refresh weekly
Filing index	<code>https://data.sec.gov/submissions/CIK{cik10}.json</code>	Per-company list of filings (form type, accession, dates). Used to know what filings exist; does not contain financial values.
All facts (primary)	<code>https://data.sec.gov/api/xbrl/companyfacts/CIK{cik10}.json</code>	SEC's pre-extracted XBRL facts , aggregated

Purpose	Endpoint	Notes
		across all of a company's 10-K / 10-Q / 8-K filings. Not the filings themselves. One JSON per company. This is V1's main data source.
Single concept (fallback)	<code>https://data.sec.gov/api/xbrl/companyconcept/CIK{cik10}/{taxonomy}/{tag}.json</code>	Same data sliced by concept. Used for targeted re-fetch when <code>companyfacts</code> has a gap for a known concept.
Raw filing index	<code>https://www.sec.gov/cgi-bin/browse-edgar?...</code>	Used only to locate the path of a specific filing's XBRL instance document when the fallback path is required.
Raw XBRL document	<code>https://www.sec.gov/Archives/edgar/data/{cik}/{accession-stripped}/{primary_doc}</code>	The XBRL instance document (XML) inside a specific filing. Used by the XBRL-XML fallback path when <code>companyfacts</code> has a gap.
8-K Item 4.02 primary document	<code>https://www.sec.gov/Archives/edgar/data/{cik}/{accession-stripped}/{primary_doc}</code>	The 8-K's primary document (HTML / iXBRL). Downloaded only when an Item 4.02 8-K is in the submissions index. Parsed narrowly to extract the affected fiscal periods. No financial values are extracted; no other 8-K item types are parsed.

7.2 Compliance

- **User-Agent:** the SEC requires a descriptive `User-Agent` of the form `AppName/Version contact@example.com`.
Source: `https://www.sec.gov/os/accessing-edgar-data` (verify against current text before release; SEC has

previously rejected requests with missing or vague User-Agent strings). The string is configurable in `config.json` and surfaced in onboarding so the user understands what is being sent.

- **Rate limit:** SEC publishes a ceiling of 10 requests/second per client. Source: <https://www.sec.gov/os/accessing-edgar-data> (re-verify before release). The client enforces a token-bucket limiter at 5 req/s by default with jittered backoff on 429/5xx; the limiter is shared across all sources (EDGAR + 8-K HTML + companyconcept) so concurrent ingestion jobs cannot stack into a violation.
- **No mass crawling:** the app fetches per-company on user action, never speculatively.

7.3 Caching and refresh

SEC's `data.sec.gov` returns `Cache-Control: max-age=0, no-cache, no-store` and **does not send** `ETag` or `Last-Modified` headers (verified against the live endpoint). HTTP-conditional revalidation is therefore not available; V1 uses a local-TTL strategy:

- `company_tickers.json` : refresh weekly (TTL 7d).
- `submissions/CIK{cik}.json` : refresh on every user-initiated company refresh; locally cached but never proactively re-fetched.
- `companyfacts/CIK{cik}.json` : re-fetched on every user-initiated refresh. The artifact is one document containing every fact ever filed for the company; SEC re-processing can change facts under a stable accession number, so the refresh path always re-parses the file and relies on `raw_fact`'s natural-key `UNIQUE` to no-op duplicates and surface genuine changes as new rows.
- Item 4.02 8-K HTML and per-filing XBRL XML documents are immutable once fetched — accession numbers never get reused.
- Historical EOD prices and FX rates: see §7.5.

The doc deliberately does **not** rely on conditional-GET semantics, so SEC tightening or relaxing cache headers won't affect correctness.

7.4 Failure modes

Condition	Response
No network	UI surfaces "offline"; ingestion buttons disabled; navigation of cached data unaffected; current-market-cap widget shows "unavailable" without affecting historical chart
429 rate limit	Exponential backoff up to 60s, then surface a user-visible warning
5xx	Same as 429 with a different message
Schema change in EDGAR JSON	Fail closed for that fact; record an <code>ingestion_event</code> with the offending payload; do not overwrite previously-good data
Missing concept for a known metric	Mark the period as having a gap; the UI shows an explicit "—"
Item 4.02 period-extraction unable to identify affected periods	Block ingestion of the 4.02 with a high-severity, user-visible <code>ingestion_event</code> . The system does not fall back to over-flagging; an unparsed 4.02 is treated as a defect to fix in the parser, not a runtime degradation.
Market-data adapter unavailable at ingestion	Persist all other ingested data; record an <code>ingestion_event</code> ; the historical-market-cap point for the affected filing is filled in on the next refresh.
Market-data adapter unavailable at runtime	Hide the current-market-cap widget; rest of dashboard unaffected.

7.5 Market-data and FX integration

Two replaceable adapters cover all non-SEC outbound traffic.

MarketDataAdapter (price data). V1 ships with a Yahoo Finance adapter as the default — chosen because it requires no API key and covers FR-050's historical-market-cap requirement out of the box. Yahoo's quote endpoints are not contract-backed; the trait exists precisely so the adapter can be swapped without touching call sites if Yahoo's surface shifts (the V18 risk register tracks this).

- **Historical EOD prices** are fetched per ticker once at first ingestion, then refreshed incrementally on each user-initiated refresh (only dates since `MAX(date)` in `historical_price` for that CIK). Cached to `<cik>/prices.json` and persisted to `historical_price` (with the ticker in effect at fetch time recorded — see M6 mitigation in the schema).
- **Current price** is fetched on demand when the dashboard's `<CurrentMarketCap>` widget mounts and the device is online. No caching; an unavailable response is surfaced gracefully without affecting the rest of the dashboard.

FXRateAdapter (currency conversion). V1 ships with an offline ECB historical-reference-rate dataset bundled in the app binary (a few MB of CSV per major currency back to ~1999). On user-initiated refresh, an incremental update is fetched from the ECB's historical-rate file (one allowlisted host, `https://www.ecb.europa.eu`). The full historical series is therefore offline-available the moment the app is installed; only updates require connectivity.

Both adapters are host-allowlisted in the Rust-side `request` client (see §13.3 / §15). The total set of non-SEC outbound destinations V1 reaches is exactly two hosts: the configured market-data provider and `ecb.europa.eu`.

8. Normalization Engine

The PRD elevates normalization to a *core architectural requirement*. The engine is implemented as a pure-functions module with the following responsibilities:

8.1 Concept mapping

A concept map associates external XBRL concepts with canonical metrics:

```
// Sketch
ConceptMap {
  revenue: [
    ("us-gaap", "Revenues"),
    ("us-gaap", "RevenueFromContractWithCustomerExcludingAssessedTax"),
    ("us-gaap", "SalesRevenueNet"),
    ("us-gaap", "SalesRevenueGoodsNet"),
    ("us-gaap", "SalesRevenueServicesNet"),
  ],
  capital_expenditures: [
    ("us-gaap", "PaymentsToAcquirePropertyPlantAndEquipment"),
    ("us-gaap", "PaymentsToAcquireProductiveAssets"),
  ],
  // ...
}
```

When multiple candidate facts could populate the same `(metric, period)`, a **resolution rule** selects exactly one as primary (`is_primary = 1`) and records the others as alternates (`is_primary = 0`). The rule order matches PRD §11.3:

1. **Prefer the most recent amendment in the supersession chain.** If a 10-K/A or 10-Q/A covers the period, its values are primary; the original 10-K / 10-Q values are kept as alternates and the amendment supersedes them via `superseded_by` (see §8.5).
2. **Prefer the original filing only when no amendment covers the period.**
3. **Prefer the canonical primary concept** within the chosen filing's facts. If that concept is absent, fall through the catalog's ordered fallback list (see the §6.2 catalog and the `total_debt` definition for an example).
4. **Prefer `xbrl_api` (companyfacts) over `xbrl_xml`** (raw XBRL fallback) when the same value appears in both — the API is the SEC's canonicalized read-side.

The decision, the inputs that contributed, and any rejected alternates are written to `ingestion_event` so the user can audit the resolution from the Diagnostics tab.

Bank-revenue fallback chain. Bank-holding companies do not tag `us-gaap:Revenues`; instead they file a combination of net-interest and non-interest items. When step 3's canonical `Revenues` candidates all return nothing for a `(cik, period_id)`, the engine falls back in order:

1. `us-gaap:Revenues` (handled by the canonical concept map above).
2. `NetInterestIncome + NoninterestIncome`.
3. `(InterestIncomeOperating - InterestExpense) + NoninterestIncome`.

The fallback runs as a separate pass after the canonical normalize step (the inputs themselves go through normal normalization first), and the resulting value is persisted to `derived_metric` with `formula_id = "bank_revenue_v1"`. The IPC `revenue_aware_series` read path unions direct revenue rows with `bank_revenue_v1` derivations, so the dashboard widget shows revenue for both industrial and bank issuers without any per-company branching in the UI. A guard skips and warns when the derived value is non-positive (bank revenue is always positive — a negative/zero result indicates a cross-restatement input mismatch).

8.2 Period reconciliation

The most error-prone area. The engine:

- **Aligns fiscal periods to the company's `fiscal_year_end`.** A company with FYE 06-30 (like Microsoft) has its FY2024 covering Jul 2023 – Jun 2024. Apple's `fiscalYearEnd` in `submissions.json` is 0926 — the last Saturday of September, which can fall anywhere in the Sep 24–30 range; this is correctly handled because each `period` row carries `start_date` and `end_date` from the filing rather than synthesizing them from a fixed-month-day. Each `period` row carries the FYE in effect at the time so historic alignments are preserved if the FYE later changes.
- **Derives `period.fiscal_year` and `period.fiscal_quarter` from the period-end date, not from the SEC `fy / fp` tags.** Each `companyfacts` fact carries the fiscal-year and fiscal-period of the FILING that disclosed the fact, not of the period the fact represents. A FY2025 10-K embedding three years of comparative income-statement data tags every embedded row with `fy=2025`; a Q1 filing's prior-year-end balance sheet is tagged `fp=Q1`. Trusting either tag silently mis-labels every period in the comparative window. The engine therefore computes `fiscal_year = compute_fiscal_year(end_date, fye_mmdd)` and, for instance, `fiscal_quarter = compute_fiscal_quarter(end_date, fye_mmdd)`. The SEC tags are read only as classification hints inside reconciliation (the `fp` letter tells "is this Q1 vs Q2 vs Q3 vs FY"), never as the authoritative period identifier.
- **Distinguishes instant from duration facts.** Balance-sheet items are stored at a point in time (`is_instant = 1`); income- and cash-flow items are stored over a duration. Mismatches are diagnostic-flagged and not silently coerced.

- **Distinguishes single-quarter from year-to-date facts that share the same `fp` tag.** SEC's companyfacts API tags both styles with the same `fp` letter — a single-quarter Q2 (~90 days) and a YTD H1 (~180 days) both carry `fp=Q2` . The engine classifies each duration fact into a span-aware **slot** combining position-in-year with span: `SingleQ1..Q4` (≤110 days), `YtdH1` (150–210 days), `Ytd9M` (240–290 days), `Fy` (≥340 days). Slot identity is what gets keyed during reconciliation, not the raw `fp` tag.
- **Handles YTD vs. quarterly reporting.** Many filings report Q3 as the 9-month YTD total. The engine *always* derives the single-quarter value when only YTD is reported ($Q3 = 9M\ YTD - H1\ YTD$; $Q2 = H1\ YTD - Q1$; $Q4 = FY - 9M$; etc.), preferring the directly reported quarter when it is available. Every derivation writes a lineage record naming the YTD inputs used. A guard refuses to persist a derived single-quarter value that turns out negative for a positive-only metric (revenue, total assets, gross profit, ...); a negative result indicates the inputs straddle a restatement and the system prefers an explicit gap to a known-wrong value.
- **Concept-consistency rule within a (metric, fiscal year).** Several canonical metrics have multiple fallback XBRL concepts whose scopes differ — `DepreciationAndAmortization` (annual-only) versus `DepreciationAmortizationAndAccretionNet` (quarterly, includes accretion), for example. Mixing them inside $Q4 = FY - 9M$ produces a negative because the accretion-inclusive 9M is larger than the accretion-free FY. The reconciler therefore picks ONE source concept per (metric, fiscal_year) and uses only that concept's facts for derivation. Selection is by slot coverage (number of distinct slots present for the concept) with ties broken by the catalog's documented priority order.
- **Handles 52/53-week fiscal calendars** (common for retailers like Costco, Apple-pre-2008, Cisco). The engine detects 53-week years via `end_date - start_date > 364 days` and sets `period.is_53_week = 1` . Year-over-year comparisons on 53-week years are explicitly flagged in the UI rather than silently averaged with 52-week years.
- **Handles fiscal-year-end changes.** When a filing's `period_of_report` indicates a different FYE than the company's previously recorded FYE, the engine inserts a high-severity, user-visible `ingestion_event` and creates new `period` rows under the new FYE. It does not retroactively rewrite historical periods. The UI surfaces a banner on the Company Dashboard for any company whose FYE has changed at any point in its history.
- **Refuses to invent periods.** If a fact's period boundaries cannot be reconciled to a `period` row (e.g., a stub period after IPO, an unusual transition period after FYE change), the fact is parked in `raw_fact` but not promoted to `normalized_fact` , and a user-visible `ingestion_event` is written. Accuracy is not traded for coverage.

8.3 Unit normalization

Each `raw_fact` records its declared unit. Storage uses the §6.2 micro-unit convention. The companyfacts JSON path receives absolute values from SEC and applies the $\times 10^6$ scaling at parse time; the XBRL XML fallback applies `decimals / unitRef` scaling first, then the same micro-unit scaling.

- Currency facts → USD micro-units (after FX conversion if non-USD source).
- Share counts → absolute integer share counts.
- Per-share ratios → micro-units of the declared unit (e.g., `USD/shares × 106`).
- **Non-USD reporting currencies** (foreign private issuers): the engine consults the company's `dei:EntityReportingCurrencyISOCode` fact and applies an FX conversion to USD using the **bundled offline ECB historical reference-rate dataset** (see §7.5; populates the `fx_rate` table). The rate used is the ECB rate for `period_end` (or the nearest prior business day if `period_end` is a non-trading day). The `normalized_fact` row records `original_value` , `original_unit` , `fx_rate_micro` , `fx_rate_source` , and `fx_rate_date` so the lineage panel can show the conversion. The dashboard always renders USD by default with an "as reported in " tooltip; users can toggle to view the original-currency series. This satisfies PRD §6.5's currency-inconsistency requirement without dropping non-USD filers and without adding any third-party dependency.

8.4 Sign normalization

Cash-flow concepts in particular use mixed sign conventions across companies (CapEx may be reported as positive payments or negative cash flow). The catalog defines the storage sign convention; the normalizer applies it deterministically and records both the original sign and the applied transform in lineage.

8.5 Restatement handling

When a 10-K/A or 10-Q/A is ingested:

1. New `filing` row marked `is_amendment = 1`, `amends = <original accession>`.
2. New `raw_fact` rows inserted (idempotent via the natural-key UNIQUE).
3. **For every (metric, period_id) the amendment re-tags:** a new `normalized_fact` row is inserted with `is_primary = 1`. The supersession chain is updated as a **linked list**: the immediate predecessor (the previously-current primary row with `superseded_by` IS NULL for the same (cik, metric, period_id)) gets its `superseded_by` set to the new row's id. Earlier rows in the chain are not modified.
4. **For every (metric, period_id) the original filing tagged but the amendment is silent on:** the original row stays `is_primary = 1`, `superseded_by` IS NULL (no implicit retraction — amendments commonly only re-tag what's being corrected, and treating omission as withdrawal would over-flag). An `amendment_coverage_gap` row is inserted recording the amendment accession, the metric, and the period. A `warn-level ingestion_event` is also written with `user_visible = 1` so the dashboard surfaces a per-period caveat (see §11.2) — the existence of an unaddressed amendment scope is never silent.
5. The triggers `trg_norm_no_cycle_update` and `trg_norm_no_cycle_insert` (see §6.3) refuse cycles on both update and insert paths.
6. **Resolution of any pending Item 4.02 announcements** for the amendment's covered periods: see §8.7 — the amendment's accession is recorded in `restatement_resolved_by` for every flagged period it covers, which clears the dashboard warning.
7. Read queries default to `WHERE is_primary = 1 AND superseded_by IS NULL`. The lineage panel walks the chain forward and backward via `idx_norm_superseded_by` to show the full restatement history.

Multi-step amendments (10-K → 10-K/A → 10-K/A2): each amendment supersedes only its immediate predecessor. The chain reads as `0 → A1 → A2`; `WHERE superseded_by IS NULL` returns A2 alone; the lineage panel walks `O ← A1 ← A2` on demand. Cycle protection prevents pathological inputs from corrupting the chain on either insert or update.

Insert-vs-supersession-update ordering. The partial unique index `idx_norm_primary_current` ON `normalized_fact` (cik, metric, period_id) `WHERE is_primary = 1 AND superseded_by IS NULL` is the database-level guarantee that there is at most one "currently primary" row per metric-period. SQLite does not support deferred uniqueness checks for partial indexes — UNIQUE is enforced per statement, not per transaction. The supersession write therefore runs in this order, inside a single transaction:

1. Idempotency probe: if a row already exists for (cik, metric, period_id, source_fact_id) (the same raw fact mapping to the same period) return its id and exit. This makes re-ingestion a no-op at this layer.
2. Demote the previous primary: `UPDATE ... SET is_primary = 0 WHERE id = prev_id`. After this step the partial index has no row for the metric-period.
3. Insert the new primary row.
4. Restore the previous row: `UPDATE ... SET is_primary = 1, superseded_by = new_id WHERE id = prev_id`. The combination of `is_primary = 1` with `superseded_by IS NOT NULL` satisfies the partial-index predicate's negation, so the row stays out of the partial index but reads correctly via the supersession-chain walk.

Doing the insert before the demote, or skipping the demote and relying on `superseded_by` alone, would fire the partial unique index. Doing the restore before the insert would lose its `superseded_by → new_id` link.

8.6 Conflict surfacing

The PRD says: *"The application must never silently discard normalization conflicts or ambiguities."*

Every place the engine has to make a choice writes an `ingestion_event`. The `level` column distinguishes `info` (routine derivations like YTD→Q3), `warn` (resolved conflicts where alternates were demoted), and `error` (unresolved cases where the fact was not promoted). The `user_visible` column flags the events that surface on the dashboard itself rather than only in the Diagnostics tab — `error` events and FYE-change banners are user-visible by default; routine `info` events stay in Diagnostics.

8.7 Item 4.02 restatement-warning handling

When an Item 4.02 8-K is identified in the submissions index (`filing.item_4_02_8k = 1`):

1. The 8-K's primary document is downloaded and saved to `<accession_no>/item_4_02.html`.
2. A dedicated parser extracts the specific fiscal periods the disclosure flags as unreliable. The parser must do this with full accuracy regardless of phrasing or structured-tag presence; an inability to identify the affected periods is a parser bug, not a runtime case to handle gracefully (see §7.4).
3. For each identified period, a `restatement_announcement` row is inserted referencing the 4.02 accession and the affected `period_id`.
4. **Resolution is tracked explicitly via the `restatement_resolved_by` join table**, not via date arithmetic. When any 10-K/A or 10-Q/A is ingested (§8.5), the persist worker iterates the open `restatement_announcement` rows for the same `cik` and inserts a `restatement_resolved_by` row for every announcement whose `affected_period_id` is covered by the amendment. (An amendment's coverage is determined by the periods for which it produced new `normalized_fact` rows in step 3 of §8.5 — a positive, observable signal, rather than guessing from form type or date.)
5. The dashboard renders a per-period warning whenever this query returns at least one row:

```
SELECT ra.id, ra.affected_period_id, ra.filed_at
FROM restatement_announcement ra
WHERE ra.cik = ?
  AND NOT EXISTS (
    SELECT 1 FROM restatement_resolved_by r
    WHERE r.restatement_announcement_id = ra.id
  );
```

The query is unambiguous, deterministic, and decoupled from the form-type / date-arithmetic fragility that the offline reviewer's M7 finding flagged.

6. **No financial values are extracted from the 8-K itself.** Restated values land via the normal `raw_fact` / `normalized_fact` / `superseded_by` path when the 10-K/A or 10-Q/A is filed and ingested.

9. Ingestion Pipeline

9.1 Stages

Discover → Download → Parse → Normalize → Persist

Each stage takes a typed input and produces a typed output; each is independently testable with golden fixtures.

Stage	Input	Output	Side effects
Discover	Ticker	CIK + filing index from <code>submissions.json</code> , with Item 4.02 8-Ks flagged	None
Download (facts)	CIK	<code>companyfacts.json</code> (primary); per-accession XBRL instance XML (fallback only)	Filesystem
Download (4.02)	Item 4.02 8-K accessions	8-K primary documents (HTML / iXBRL)	Filesystem
Download (prices)	CIK + new date range	Historical EOD prices since last refresh	Filesystem + DB
Parse	<code>companyfacts.json</code> , fallback XBRL XML, 8-K Item 4.02 documents	<code>RawFact</code> records + <code>RestatementAnnouncement</code> records	None
Normalize	<code>RawFacts</code> + <code>ConceptMap</code>	<code>NormalizedFacts</code> + <code>Diagnostics</code>	None
Persist	<code>NormalizedFacts</code> + <code>RawFacts</code> + <code>RestatementAnnouncements</code> + <code>Prices</code> + <code>Diagnostics</code>	Persisted state	DB

Normal-path traffic. For a typical ingestion of a company with no Item 4.02 history, V1 makes three HTTP requests: one to `data.sec.gov/submissions/CIK*.json`, one to `data.sec.gov/api/xbrl/companyfacts/CIK*.json`, and one to the market-data adapter for historical prices. When Item 4.02 8-Ks exist in the submissions index, one additional request per 4.02 fetches its primary document. Per-filing XBRL XML requests are made *only* on the fallback path. 10-K / 10-Q HTML / iXBRL documents are never fetched.

9.2 Concurrency model

- Stages run as `tokio` tasks within a per-ingestion job.
- Network I/O (Discover, Download) is rate-limited globally via a shared token bucket so multiple ingestion jobs cannot exceed SEC rate limits.
- DB writes happen on a single writer task that owns the SQLite write connection; readers use a separate pool. This avoids `SQLITE_BUSY` while WAL is in use.
- Progress is published over a Tauri event channel (`ingestion://progress/<job_id>`) for the UI to render progress bars.

9.3 Idempotency

- Ingestion is keyed by `accession_no`. Re-running ingestion for an already-persisted filing is a no-op.
- A partially-completed job can be resumed: the persist stage records a checkpoint per filing, and Discover skips filings already at the latest stage.

9.4 Failure semantics

Per the PRD, partial ingestion is acceptable as long as it is visible. The pipeline:

- **Continues** when a single filing fails to parse — other filings still produce data.
- **Halts** the job on the following SQLite-level signals: `SQLITE_CORRUPT` , `SQLITE_NOTADB` , `SQLITE_FULL` , `SQLITE_IOERR_*` . The persist worker translates these into a top-level `IngestionError::DatabaseUnhealthy(code)` ; the UI surfaces a non-dismissable error and disables further ingestion until a manual `PRAGMA integrity_check` passes.
- **Records every failure** as an `ingestion_event` row so the user can see what was skipped.

10. Derived Metric Engine

V1 ships a small fixed catalog of formulas. Each formula is a Rust function that declares the canonical metrics (and, where relevant, the historical-price inputs) it consumes, and returns either a value with a lineage record or `is_complete = 0` with the missing-input list.

V1's registered formulas:

Formula id	Output metric	Inputs	Notes
<code>fcf_v1</code>	(derived per period)	<code>net_income</code> , <code>depreciation_amortization</code> , <code>capital_expenditures</code>	FCF = NI + D&A – CapEx (CapEx is sign-normalized positive at storage). All inputs and the result are in USD micro-units; integer arithmetic, exact.
<code>total_debt_v1</code>	<code>total_debt</code>	<code>long_term_debt</code> , <code>current_debt</code>	Sum, with the per-input fallback chain documented in §6.2. Read-time derivation (the IPC layer joins each input's most-recent primary value per period and sums on the fly), not persisted, so a superseded <code>current_debt</code> cannot leave a stale <code>total_debt</code> cached.
<code>gross_profit_v1</code>	<code>gross_profit</code>	<code>revenue</code> , <code>cost_of_revenue</code>	Used only when <code>gross_profit</code> is not directly tagged in <code>companyfacts</code> . Read-time derivation, same rationale as <code>total_debt_v1</code> .
<code>capital_expenditures_v1</code>	<code>capital_expenditures</code>	<code>property_plant_and_equipment_net</code> (this period and prior), <code>depreciation_amortization</code>	Fallback when no explicit cash-flow CapEx is tagged: $\text{CapEx} \approx \Delta \text{PP\&E}_{\text{net}} + \text{D\&A}$. Read-time derivation. Some bank/financial-services filers omit explicit CapEx; without this fallback the dashboard widget would render a spurious zero or gap for them.

Formula id	Output metric	Inputs	Notes
bank_revenue_v1	revenue (alternate path)	net_interest_income , noninterest_income , interest_income_operating , interest_expense	Resolution chain (§8.1): step 3 = NetInterestIncome + NoninterestIncome ; step 4 = (InterestIncomeOperating - InterestExpense) + NoninterestIncome . Run at ingest time and persisted to derived_metric . The IPC revenue_aware_series reader unions this against direct revenue rows so bank issuers render correctly without any UI branching. A non-positive derived value is skipped + warned (bank revenue is always positive).
historical_market_cap_v1	historical_market_cap	historical_price[filed_at] , shares_outstanding_basic	Computed once per filing at ingestion: (close_micro × shares_outstanding) / 1 — both factors are in micro-units / absolute integers, the result is in USD micro-units. Persisted to derived_metric ; offline-available.
current_market_cap_v1	current_market_cap	live price, shares_outstanding_basic (latest)	"Latest" = shares_outstanding_basic from the most recent primary, non-superseded normalized_fact : WHERE is_primary = 1 AND superseded_by IS NULL ORDER BY period_id DESC LIMIT 1 . When this value is more than 120 days old, the dashboard widget renders a "shares as of " caveat. Computed on demand; not persisted; widget shows an explicit unavailable state when the live price source is offline.

If any input is missing, the result is `is_complete = 0` with no value, and the UI displays a gap rather than a fabricated number. This satisfies FR-030 / FR-031 / FR-032 and the PRD principle of "explicit gaps, no fabricated estimates."

The engine is built so V2's user-defined formulas can plug in via a registry — but V1 only registers compile-time functions.

11. UI Architecture

11.1 Routes

Route	Purpose
<code>/</code>	Home: saved companies, add ticker, refresh status
<code>/c/:ticker</code>	Company dashboard
<code>/c/:ticker/statement/:kind</code>	Income / Balance / Cash flow tables
<code>/c/:ticker/metric/:metric</code>	Drill-down: per-metric history + lineage
<code>/c/:ticker/diagnostics</code>	Ingestion diagnostics & data-quality view

11.2 Component composition

- **Dashboard** = `<RestatementBanner/>` + `<FyeChangeBanner/>` + `<SummaryWidgets/>` + `<ChartGrid/>` + `<StatementsTable/>`.
- **RestatementBanner** renders whenever §8.7's resolution query returns at least one period for the current company. Names the affected period(s) and the date the 4.02 was filed. Clears period-by-period as covering amendments are ingested.
- **FyeChangeBanner** renders for any company whose fiscal-year-end has changed at any point in its ingested history.
- **SummaryWidgets** displays revenue, net income, cash, total debt, FCF, and historical market cap as the latest reported annual value with a sparkline of recent quarters. A `<CurrentMarketCap/>` sub-widget is rendered when the live price source is online; when offline or the source is down, it shows an explicit "current market cap unavailable" state without affecting the rest of the dashboard.
- **ChartGrid** exposes Annual / Quarterly toggle (FR-051), default 10y, configurable to 20y. Charts use ECharts via `echarts-for-react` with chart-instance reuse (`notMerge: false` , `lazyUpdate: true`) and `useMemo` on series props so re-renders do not stall.
- **StatementsTable** is a virtualized table (TanStack Table) with row drill-down → `/c/:ticker/metric/:metric`. Rows carry per-row indicators for two distinct conditions:
 - **"Unreliable"** (red) — the period is flagged by an unresolved Item 4.02 (§8.7).
 - **"Partially amended"** (amber) — an amendment for this period exists but did not re-tag this concept (an `amendment_coverage_gap` row exists; see §8.5 step 4). Click expands to show which amendment and which concepts were and weren't covered. Both indicators are present when both conditions hold.
- **Lineage panel** is a side drawer that shows: filing accession, form type, filing date, XBRL concept, raw value, sign transform, supersession chain, and any FX conversion applied for non-USD filers (FR-060 / FR-061 / FR-062).

11.3 State and data flow

- All data is fetched via Tauri IPC commands wrapped as TanStack Query hooks.
- The query cache is the de-facto in-memory state; there is no Redux.
- Mutations (add company, refresh) invalidate relevant queries and trigger re-fetch.
- Long-running jobs (ingestion) emit progress events; the UI subscribes via Tauri event listeners.

11.4 Design system

- Density: ~13 px base font; tabular numerals for all financial values.

- Color: a constrained palette (neutrals + one accent) with explicit positive/negative colors for variances. No gradients, no shadows beyond elevation-1.
- Typography: a single sans-serif (system stack: SF Pro on macOS) plus a mono stack for raw values in the lineage panel.
- Loading states: skeleton placeholders for tables; explicit "missing data" markers (PRD §10.5 error visibility).

12. Concurrency, Performance, and Caching

12.1 SQLite tuning

```
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
PRAGMA temp_store = MEMORY;
PRAGMA mmap_size = 268435456; -- 256 MB
PRAGMA foreign_keys = ON;
```

WAL is essential so the UI can read while ingestion writes.

On `synchronous = NORMAL`. This is the SQLite-recommended default for desktop applications. It is durable across application crashes (which is what PRD §7.1 explicitly requires); it is **not** durable against power loss / OS crash without `synchronous = FULL`, which roughly halves write throughput during ingestion. We choose `NORMAL` because (a) all canonical data is re-ingestable from EDGAR — the worst case after an OS-crash is at most one transaction's worth of work to re-fetch — and (b) ingestion checkpointing in §9.3 makes any lost transaction recoverable on the next refresh.

On `PRAGMA foreign_keys` and migrations. The pragma is per-connection, not stored on disk, and cannot be toggled inside a multi-statement transaction (verified against the SQLite docs). Refinery wraps each migration in a transaction. **V1 uses additive migrations only** (`CREATE TABLE`, `CREATE INDEX`, `ADD COLUMN`); no `DROP COLUMN` or table-rebuild patterns. If a future schema evolution requires a rebuild, it will run outside refinery's transaction model on its own connection with `foreign_keys = OFF`, document the exception explicitly in the migration file, and re-enable `foreign_keys = ON` afterward.

12.2 Connection model

- One **write** connection, owned by the persist worker.
- A **read pool** of 4 connections shared across IPC handlers.

12.3 Query patterns

- The dashboard's hot path is `SELECT metric, period_id, value FROM normalized_fact WHERE cik = ? AND is_primary = 1 AND superseded_by IS NULL ORDER BY period_id` — covered by the partial unique index `idx_norm_primary_current`.
- All frequent reads are bounded by `cik`; an index on `(cik, ...)` covers them.
- The lineage panel's "what supersedes me" walk is the inverse: `SELECT id FROM normalized_fact WHERE superseded_by = ?` — covered by an additional non-unique index on `superseded_by` (added in the same migration as the table).

12.4 Caching

- Filing metadata: in DB, `last_refreshed` per company.
 - Derived metrics: persisted in `derived_metric`; recomputed only when an input changes.
 - UI: TanStack Query with a 5-minute stale time for IPC results inside a session.
-

13. Observability

13.1 Logging

- `tracing` with a JSON layer to a rotating file under `~/Library/.../logs/`.
- A console layer when launched with `RUST_LOG=debug`.
- Log levels: `info` for stage transitions, `warn` for fallbacks and conflicts, `error` for skipped data.
- **Rotation**: daily rotation; retain at most **30 days** of files OR **100 MB** total, whichever is reached first. Older files are deleted on app start.

13.2 In-app diagnostics

Every interesting decision the system makes during ingestion writes a row to `ingestion_event`. The **Diagnostics** tab in the company dashboard renders this table with filters. This is what makes the PRD's "transparency" requirement concrete.

13.3 No telemetry

Nothing is sent off the device. The app does not ping a vendor URL, does not check for updates, and does not phone home. Outbound traffic is gated by **three independent mechanisms** (defense-in-depth — verified against the live Tauri 2 docs):

1. **Tauri 2 capabilities** (declared in `src-tauri/capabilities/*.json`) restrict which `#[tauri::command]` handlers and plugin APIs the WebView can invoke. This is a *positive grant* model on commands, not a network filter.
2. **WebView Content-Security-Policy** (configured via the `csp` field in `tauri.conf.json`, specifically `connect-src`) restricts which hostnames the frontend can reach with `fetch()` / XHR. CSP is what actually blocks frontend network calls to arbitrary hosts; capabilities alone do not.
3. **Rust-side host allowlist** in the `request` client builder restricts which hostnames the Rust core can call out to: `www.sec.gov`, `data.sec.gov`, the configured market-data adapter's host, and `www.ecb.europa.eu`. Any other URL fails closed at the HTTP layer regardless of code path. Tauri's permission system does not govern Rust-side `request` calls — the host allowlist is what enforces this layer.

All three layers are required; none is sufficient on its own.

14. Error Handling

14.1 Layered errors

- **Source layer**: `SourceError` with variants for HTTP status, parse failures, schema violations.
- **Pipeline layer**: `IngestionError` wraps `SourceError` and adds stage context.
- **Service layer**: `AppError` is the top-level enum exposed across IPC; it carries a stable `code` string for the UI to switch on, plus a localized message.

14.2 IPC error contract

Every Tauri command returns `Result<T, AppError>`. The frontend's typed client maps `AppError.code` to user-visible messaging; the underlying message and detail are surfaced in a "Show details" affordance for advanced users.

14.3 Partial success

A common case: ingestion succeeds for 18 of 20 filings. The IPC response is `Ok(IngestionSummary { succeeded, skipped: [...] })` and the UI shows a non-blocking notice with a link to Diagnostics.

15. Security and Privacy

Concern	Mitigation
Outbound traffic	Tauri 2 capabilities (JS→Rust commands) + WebView CSP <code>connect-src</code> (frontend <code>fetch /XHR</code>) + <code>request host allowlist</code> (Rust outbound) — all three required, see §13.3
Outbound destinations (V1 total)	<code>www.sec.gov</code> , <code>data.sec.gov</code> , the configured market-data adapter (Yahoo Finance by default), <code>www.ecb.europa.eu</code>
Telemetry	None — zero analytics, zero remote config
Local data integrity	SQLite WAL + <code>synchronous = NORMAL</code> (durable across app crashes; not OS-crash — see §12.1 trade-off note); <code>PRAGMA integrity_check</code> on startup
Filesystem permissions	Data stored under <code>~/Library/Application Support/<bundle>/</code> ; App Sandbox enabled with <code>com.apple.security.network.client</code> for outbound HTTP
Code signing	Developer ID-signed and notarized for distribution; required before any external build
Dependencies	Minimal set; <code>cargo audit</code> in CI; <code>pnpm audit</code> for the React side
Secrets	None stored; SEC has no auth; the User-Agent contact email is user-supplied

16. Build, Packaging, and Distribution

- **Local development:** `pnpm tauri dev`.
- **Production build:** `pnpm tauri build` produces `.dmg` and `.app`.
- **Code signing & notarization:** Developer ID-signed and notarized via Apple's notarytool. App Sandbox enabled. Required before any external build is distributed.
- **Distribution channel (V1):** developer-distributed `.dmg` from a personal-website download. **No auto-update mechanism in V1.** App Store distribution and managed-update channels are V2+.
- **Unit tests:** `cargo test` for Rust, `vitest` for the React side.
- **Integration tests:** golden-fixture tests that ingest a date-pinned, checked-in copy of an EDGAR `companyfacts` JSON for sample companies (Apple as the canonical fixture; one bank/insurer/REIT/foreign-private-issuer as edge-case fixtures). Fixture-pinning date is recorded next to each fixture so future regressions are obvious. Item 4.02 parser tests use a corpus of past Item 4.02 8-Ks across multiple phrasings.
- **CI (GitHub Actions, post-V1):** macOS runner, lint + test + bundle.

17. Future Extensibility

The architecture has explicit seams for the V2/V3 directions in the PRD:

Future feature	Seam
Multi-company comparison	Repository queries are already keyed by <code>cik</code> ; UI introduces a new layer that joins multiple companies. No schema change.
User-defined formulas	The derived-metric engine already accepts a registry; V2 adds a parser + sandboxed evaluator.
Peer benchmarking	A <code>peer_group</code> table joins companies; the metric service gains a <code>for_peers</code> API.
Export	A new <code>export/</code> module reads from <code>normalized_fact</code> and <code>derived_metric</code> and writes CSV/Excel/PDF. The data is unchanged.
Plugin SDK	The stable Rust traits (<code>MetricProvider</code> , <code>SourceAdapter</code> , <code>MarketDataAdapter</code>) are already what the in-tree modules implement. The eventual loading mechanism (dynamic library, WebAssembly, or in-process Rust extension shipped as part of a plugin bundle) is a V3 design choice with non-trivial security and code-signing implications and is not decided in V1.
Local LLM	The lineage and normalization layers already produce structured records that can be fed as context.

The boundary that matters most is keeping `raw_fact` separate from `normalized_fact` . Every future feature will benefit from the audit trail this provides.

18. Risks and Open Questions

Risk	Likelihood	Mitigation
EDGAR companyfacts API changes its schema	Low	Fail closed per fact; raw XBRL fallback; integration tests on golden fixtures
Concept-map coverage is incomplete for some industries (banks, insurers, REITs, foreign private issuers)	High	V1 ships a vetted catalog for non-financials, banks, insurers, REITs, and FPIs; per-industry golden fixtures gate releases
Item 4.02 parser fails to identify affected periods on a novel phrasing	Medium	Block ingestion for the offending 4.02 with a high-severity user-visible event; add the phrasing to the test corpus and ship a parser fix. No over-flagging fallback.
Companies with non-USD reporting currency	Medium	Ingest fully; store both original-currency and USD-converted values; record FX rate and source in lineage
Restatement chains spanning >1 amendment	Medium	Linked-list <code>superseded_by</code> chain; cycle-protection triggers on both INSERT and UPDATE paths; lineage panel walks the chain via <code>idx_norm_superseded_by</code>
Amendment is silent on a concept the original tagged	Medium	Original stays primary; <code>amendment_coverage_gap</code> row inserted; per-period dashboard caveat surfaced (§8.5 step 4, §11.2) — never silent
User runs ingestion for many companies in parallel and hits SEC rate limits	Medium	Global token bucket shared across all sources; queue rather than reject
SQLite corruption from disk-full	Low	WAL + <code>PRAGMA integrity_check</code> on startup; APFS copy-on-write helps

Risk	Likelihood	Mitigation
Ticker→CIK lookup ambiguity (multiple share classes)	Medium	Disambiguation UI when the lookup returns >1 candidate
Market-data adapter (Yahoo Finance) becomes unreliable	Medium	<code>MarketDataAdapter</code> trait keeps the call sites unchanged; swap provider via configuration; degraded UI when adapter is unavailable

Open questions

The architecture's previous open-questions list has been resolved by the V1 Q&A:

1. ~~Reporting currency support~~ → V1 ingests non-USD filers fully (§8.3).
2. ~~Quarterly derivation policy~~ → Always derive single-quarter values from YTD when a directly reported quarter is unavailable (§8.2).
3. ~~Market cap source~~ → Yahoo Finance via the `MarketDataAdapter` trait; historical EOD persisted, current value live-only (§7.5, §10).
4. ~~Code signing timing~~ → Required before any external build (§16).

No new open questions remain at the architecture-doc altitude. Implementation-level choices are tracked in code reviews and ADRs.

19. Appendices

A. Sample SEC URLs (for reference)

- Apple ticker→CIK: `https://www.sec.gov/cgi-bin/browse-edgar?action=getcompany&CIK=AAPL&type=10-K&dateb=&owner=include&count=40`
- Apple submissions: `https://data.sec.gov/submissions/CIK0000320193.json`
- Apple company facts: `https://data.sec.gov/api/xbrl/companyfacts/CIK0000320193.json`
- Apple revenues concept: `https://data.sec.gov/api/xbrl/companyconcept/CIK0000320193/us-gaap/Revenues.json`

B. Glossary

- **CIK** — Central Index Key, SEC's unique 10-digit company identifier.
- **XBRL** — eXtensible Business Reporting Language; the XML-based standard used in SEC structured filings.
- **Taxonomy** — a published vocabulary of XBRL concepts (e.g., `us-gaap`, `ifrs-full`).
- **Concept** — a single tagged datapoint within a taxonomy (e.g., `us-gaap:Revenues`).
- **Accession number** — SEC's unique identifier for a single filing submission.
- **Form 10-K / 10-Q** — annual / quarterly report.
- **/A suffix** — amendment (e.g., `10-K/A`).
- **Instant vs. duration** — XBRL distinction between point-in-time facts (balance sheet) and over-period facts (income / cash flow).
- **Fiscal period** — a company-defined accounting period; may not align with the calendar year.
- **Item 4.02 8-K** — a Form 8-K filing under Item 4.02, "Non-Reliance on Previously Issued Financial Statements." The SEC requires this filing when management concludes that previously filed financial statements should no longer be relied upon.

- **Restated** — corrected version of a previously filed financial statement, typically published in a 10-K/A or 10-Q/A amendment.
- **52/53-week fiscal year** — a fiscal calendar (common for retailers) where each year ends on a fixed weekday, producing a 52- or 53-week year depending on calendar drift.
- **MD&A** — Management's Discussion and Analysis, the prose section of a 10-K/10-Q where management explains the financial results. Out of V1 scope as a content source.

C. Document conventions

- Code blocks marked `// Sketch` are illustrative, not final API.
- Schema DDL in §6.3 is the canonical baseline for migration `0001_initial.sql`.
- Any deviation from the catalog or DDL during implementation must be captured in a follow-up ADR under `docs/adr/`.